

TUGAS MINGGU KETIGA MATA KULIAH KRIPTOGRAFI

KELOPOK III

Dosen pengampu:

Nizam Albar, S.T., M.T.



Nama Anggota Kelompok :

Muhammadon : (230705077)
Muhammad Ndhar : (230705083)
Nailul Amal : (230705083)
T.M.Arifian : (230705083)
Arif Alfarishi : (230705083)

**PRODI TEKNOLOGI INFORMASI FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISTAL NEGERI AR-RANIRY BANDA ACEH
PERIODE 2025/2026 GENAP**

CODE

kode `key_space_analysis.py` digunakan untuk menganalisis tingkat keamanan suatu algoritma kriptografi (seperti Caesar dan Vigenere Cipher) berdasarkan kekuatan kuncinya.

Berikut adalah 3 fungsi utama dari kode tersebut:

- Menghitung Ruang Kunci (Key Space): Menghitung seberapa banyak total kemungkinan kombinasi kunci yang bisa digunakan. Semakin panjang kuncinya, semakin besar ruang kuncinya .
- Mengestimasi Waktu Brute Force: Menghitung secara matematis berapa lama waktu yang dibutuhkan oleh komputer (dengan asumsi kecepatan tertentu, misal 1 juta percobaan/detik) untuk menebak kunci sampai ketemu melalui skenario rata-rata (setengah dari total ruang kunci).
- Mengukur Kecepatan Aktual: Melakukan serangan brute force nyata pada ciphertext sandi Caesar (mencoba kunci 1-25) untuk mengukur berapa milidetik waktu riil yang dihabiskan komputer Anda dan berapa kecepatan aslinya.

`key_space_analysis.py`

```
# =====
# key_space_analysis.py - Analisis Ruang Kunci & Kompleksitas
# Tugas Besar Kriptografi - Minggu 3
# =====

import time
import sys
import os

# Memetakan path ke folder minggu_1 agar kelas SandiCaesar bisa diimpor
# Mengasumsikan struktur root/minggu_1 dan root/minggu_3 sejajar
path_minggu_1 = os.path.abspath(os.path.join(
    os.path.dirname(__file__), '..', 'minggu_1'))
sys.path.append(path_minggu_1)

try:
    from caesar_cipher import SandiCaesar
except ImportError:
    print("Error: Gagal mengimpor. Pastikan 'caesar_cipher.py' ada di
folder 'minggu_1'.")
    sys.exit(1)
```

```

class KeySpaceAnalyzer:
    """Kelas untuk menganalisis ruang kunci kriptografi dan mengukur
    performa brute force."""

    def __init__(self, percobaan_per_detik: float = 1_000_000):
        self.percobaan_per_detik = percobaan_per_detik

    def hitung_ruang_kunci(self, algoritma: str, panjang_kunci: int) ->
int:
        """Menghitung total kemungkinan ruang kunci berdasarkan
        algoritma."""
        if algoritma.lower() == 'caesar':
            return 25
        else:
            return 26 ** panjang_kunci # 26^L kemungkinan

    def format_estimasi_waktu(self, detik: float) -> str:
        """Mengubah durasi detik menjadi string format waktu yang
        manusiawi."""
        if detik < 60:
            return f"{detik:.6f} detik"
        elif detik < 3600:
            return f"{detik/60:.2f} menit"
        elif detik < 86400:
            return f"{detik/3600:.2f} jam"
        elif detik < 31_536_000:
            return f"{detik/86400:.2f} hari"
        else:
            return f"{detik/31_536_000:.2e} tahun"

    def analisis_ruang_kunci(self, algoritma: str, panjang_kunci: int) ->
None:
        """Menghitung ukuran ruang kunci dan mencetak estimasi waktu brute
        force."""
        ruang_kunci = self.hitung_ruang_kunci(algoritma, panjang_kunci)
        rata_rata = ruang_kunci / 2
        detik = rata_rata / self.percobaan_per_detik

        print(f'Algoritma          : {algoritma.upper()}')
        print(f'Panjang kunci       : {panjang_kunci} karakter')
        print(f'Ruang kunci          : {ruang_kunci:,} kemungkinan')
        print(f'Rata-rata percobaan: {rata_rata:,.0f}')
        print(f'Estimasi waktu        : {self.format_estimasi_waktu(detik)}')
        print('=' * 52)

    def ukur_waktu_aktual(self, ciphertext: str) -> None:
        """Mengukur waktu nyata yang dibutuhkan untuk brute force

```

```

Caesar."""
    mulai = time.perf_counter()

    # Looping mencoba kunci dari 1 sampai 25
    for k in range(1, 26):
        # Membuat objek baru untuk setiap kunci baru sesuai kontrak
__init__ SandiCaesar
        sandi = SandiCaesar(pesan=ciphertext, kunci=k)
        # Memanggil method dekripsi bawaan dari class SandiCaesar
        _ = sandi.dekripsi()

    selesai = time.perf_counter()

    durasi = selesai - mulai
    kecepatan = 25 / durasi

    print(f'Waktu aktual brute force : {durasi*1000:.4f} ms')
    print(f'Kecepatan aktual          : {kecepatan:,.0f}
percobaan/detik')

if __name__ == '__main__':
    # Instansiasi objek analyzer dengan kemampuan 1.000.000
percobaan/detik
    analyzer = KeySpaceAnalyzer(percobaan_per_detik=1_000_000)

    print('ANALISIS RUANG KUNCI & ESTIMASI WAKTU BRUTE FORCE')
    print('=' * 52)

    # Memanggil method analisis dari objek
    analyzer.analisis_ruang_kunci('caesar', 1)
    analyzer.analisis_ruang_kunci('vigenere', 4)
    analyzer.analisis_ruang_kunci('vigenere', 8)
    analyzer.analisis_ruang_kunci('vigenere', 16)

    print('PENGUKURAN WAKTU AKTUAL:')
    analyzer.ukur_waktu_aktual('KHOOR ZRUOG WKLIV LV FUBSWR FODVV')

```

Ouput :

```
python3 key-space-ananlisis.py
ANALISIS RUANG KUNCI & ESTIMASI WAKTU BRUTE FORCE
=====
Algoritma      : CAESAR
Panjang kunci   : 1 karakter
Ruang kunci     : 25 kemungkinan
Rata-rata percobaan: 12
Estimasi waktu  : 0.000013 detik
=====
Algoritma      : VIGENERE
Panjang kunci   : 4 karakter
Ruang kunci     : 456,976 kemungkinan
Rata-rata percobaan: 228,488
Estimasi waktu  : 0.228488 detik
=====
Algoritma      : VIGENERE
Panjang kunci   : 8 karakter
Ruang kunci     : 208,827,064,576 kemungkinan
Rata-rata percobaan: 104,413,532,288
Estimasi waktu  : 1.21 hari
=====
Algoritma      : VIGENERE
Panjang kunci   : 16 karakter
Ruang kunci     : 43,608,742,899,428,874,059,776 kemungkinan
Rata-rata percobaan: 21,804,371,449,714,439,094,272
Estimasi waktu  : 6.91e+08 tahun
=====
PENGUKURAN WAKTU AKTUAL:
Waktu aktual brute force : 0.3372 ms
Kecepatan aktual         : 74,144 percobaan/detik
```

dari Output Kode Ini: Kode ini membuktikan secara ilmiah bahwa sandi Caesar sangat lemah karena kuncinya bisa ditebak dalam hitungan milidetik, sedangkan sandi Vigenere dengan kunci panjang (misal 16 karakter) sangat aman karena butuh waktu miliaran tahun bagi komputer biasa untuk memecahkannya lewat *brute force*

Analisa Code

Saat objek dibuat dengan perintah `analyzer =`

`KeySpaceAnalyzer(percobaan_per_detik=1_000_000)`, program menyiapkan spesifikasi perangkat atau kekuatan komputasi bayangan. Nilai `1_000_000` disimpan sebagai standar bahwa komputer simulasi ini mampu menguji 1 juta kunci setiap detiknya.

2. Alur Fungsi Analisis Matematis (`analisis_ruang_kunci`)

Fungsi ini bekerja murni menggunakan perhitungan rumus matematika (teoretis) tanpa melakukan peretasan secara nyata. Langkahnya meliputi:

- **Menghitung Total Kemungkinan:** Program memeriksa jenis algoritma. Jika `caesar`, ruang kuncinya langsung ditetapkan **25** (karena huruf A-Z hanya bisa digeser 1 hingga 25 kali). Jika `vigenere`, program menggunakan rumus alfabet pangkat panjang kunci:
$$26^{\text{panjang_kunci}}$$
- **Menghitung Skenario Rata-rata:** Dalam teori peretasan, peretas tidak perlu mencoba seluruh kunci untuk menemukan yang benar. Secara statistik, kunci biasanya ditemukan pada **setengah (50%)** dari total ruang kunci (`ruang_kunci / 2`).
- **Estimasi Waktu:** Total rata-rata percobaan tadi dibagi dengan kemampuan komputer (`percobaan_per_detik`). Hasil detiknya kemudian dikonversi ke satuan yang mudah dibaca manusia (detik, menit, jam, hari, atau tahun) lewat bantuan method `format_estimasi_waktu`.

3. Alur Fungsi Pengukuran Riil (`ukur_waktu_aktual`)

Berbeda dengan fungsi sebelumnya, fungsi ini melakukan simulasi serangan *brute force* yang **sebenarnya** pada komputer Anda.

- **Pencatatan Waktu Awal:** Program merekam waktu presisi internal komputer menggunakan `time.perf_counter()`.
- **Proses Perulangan (*Looping*):** Program melakukan perulangan `for` sebanyak 25 kali (mewakili kunci 1 sampai 25). Pada setiap putaran, program:
 1. Membuat objek baru dari `SandiCaesar` dengan memasukkan teks rahasia (*ciphertext*) dan kunci `k`.
 2. Memanggil fungsi `.dekripsi()` milik `SandiCaesar` untuk mencoba membongkar teks tersebut.
- **Pencatatan Waktu Akhir:** Setelah perulangan selesai, program mencatat waktu akhir dan menghitung selisihnya (`selesai - mulai`) untuk mendapatkan durasi nyata dalam milidetik (ms).

- **Menghitung Kecepatan Asli:** Program membagi 25 percobaan dengan durasi waktu aslinya untuk mengetahui berapa nilai performa kecepatan *brute force* riil dari komputer Anda saat itu.